```
$ git push → https://tu-app.workers.dev
```

Del código a producción

Desplegá una app web completa usando solo planes gratuitos.

Taller práctico · JCSA

QUÉ VAMOS A CONSTRUIR

Una lista de tareas con login, funcionando en internet

Login

Cada persona ve solo sus tareas.

Datos reales

Las tareas viven en Postgres.

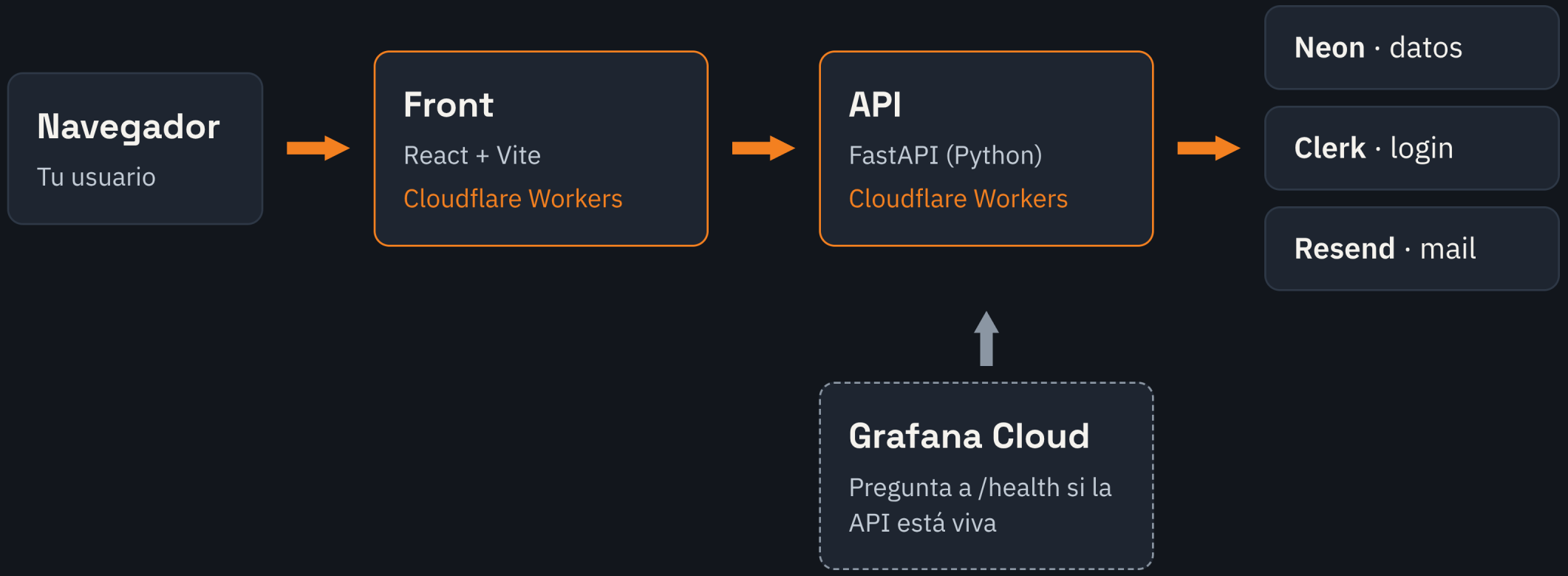
Mails

Un mail de bienvenida de verdad.

Monitoreo

Sabés si tu API está viva.

Cómo encaja todo



LA IDEA CENTRAL

El mismo código, dos entornos

	Local · tu compu	**Producción · Cloudflare**
Se levanta con	pywrangler dev · npm run dev	pywrangler deploy · push a main
Configuración	.dev.vars y .env	Secretos y variables de build
Servicios	Simulados hasta que los conectes	Siempre reales
Sirve para	Probar rápido, sin cuentas	Que lo use cualquiera

BLOQUES 1 Y 2 · FRONT Y API

1

Cloudflare Workers

Código que corre en la red de Cloudflare, cerca de cada usuario, sin servidores que administrar.

En el taller

Dos Workers: uno sirve el front y otro corre la API en Python.

Por qué

Deploy con un push, URL pública al instante y plan gratuito.

Vas a usar

```
wrangler.jsonc
npx wrangler deploy
*.workers.dev
```

BLOQUE 2 · API

FastAPI

2

Framework de Python para APIs: cada ruta es una función y los datos se validan solos.

En el taller

La API de tareas: listar, crear, marcar y borrar, siempre con login.

Por qué

Poco código, muy legible y con documentación automática.

Vas a usar

```
@app.get("/health")
uv sync
uv run pywrangler dev
```

BLOQUE 2 · BASE DE DATOS

Neon

2

Postgres serverless: una base real que se suspende sola cuando nadie la usa.

En el taller

Una tabla de tareas. La API le habla por HTTP, sin driver.

Por qué

Postgres de verdad, sin instalar nada, y SQL desde el navegador.

Vas a usar

`SQL Editor`
`schema.sql`
`NEON_CONNECTION_STRING`

BLOQUE 3 · LOGIN

Clerk

3

Login como servicio: pantallas de ingreso, usuarios y sesiones ya resueltos.

En el taller

El front muestra el login y la API verifica el token en cada pedido.

Por qué

Hacer login seguro a mano es difícil. Mejor delegarlo.

Vas a usar

`Publishable key`
`Frontend API URL`
`JWT · JWKS`

BLOQUE 4 · MAILS

Resend

4

Una API para enviar mails transaccionales con una sola llamada HTTP.

En el taller

El mail de bienvenida. Sin dominio propio, llega solo a tu mail.

Por qué

Mandar mails que no caigan en spam es un oficio aparte.

Vas a usar

```
RESEND_API_KEY
secret put
onboarding@resend.dev
```

BLOQUE 5 · MONITOREO

Grafana Cloud

5

Observabilidad: saber si tu app funciona antes de que te avise un usuario.

En el taller

Un check que consulta /health cada 2 minutos desde 2 ciudades.

Por qué

Ver disponibilidad y latencia desde afuera, como un usuario.

Vas a usar

```
Synthetic Monitoring
GET /health
{"ok":true}
```

Agenda · 2 h 20

#	Min	Qué hacemos	Listo cuando…
0	15	Arquitectura y cuentas	Todos con el repo corriendo en local
1	30	Front en Cloudflare	La app abre en *.workers.dev
2	30	API + Neon	Tareas en Neon y /health online
3	25	Login con Clerk	Cada usuario ve solo sus tareas
4	15	Mail con Resend	Llega el mail de bienvenida
5	15	Monitoreo con Grafana	El check de /health en verde
6	10	Cierre y dudas	—

Tu app ya está en internet.

Para seguir

- Un dominio propio
- CORS solo para tu front
- Tests y CI con GitHub Actions
- Tu propio proyecto

`github.com/tinxo/taller-jcsa-2026`